claude-windows / 6a82308f-69d6-43a8-9ea4-d8eaf497253d

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\subagents\workflows\wf_3d536535-241\agent-a074b04b01b78b65d.jsonl`
- SHA-256: `c27cc8344c6772e87837667f81402ca5e2a9b252baa1417a51aa76c558923d9a`
- Source modified: `2026-06-16T03:25:56+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_3d536535-241`
- session_id: `6a82308f-69d6-43a8-9ea4-d8eaf497253d`

Transcript

1. user (2026-06-16T03:24:33.250Z)

Adversarial Claim Verifier (voter 1/3)

Be SKEPTICAL. Try to REFUTE this claim. ?2/3 refutations kill it.

Research question

Research question: **What concrete, recent (2022?2025) techniques and evaluation practices should a local, commercially-licensed badminton rally-detector adopt to close the quality gap with a strong cloud VLM (Gemini), given a tiny labeled set (~6 ? ~16 golden videos)?** Produce an adversarially-verified, cited report organized around the four dimensions below. Prefer specific named methods, papers (arXiv/venue), and reported numbers over generic advice; flag anything you cannot verify.

CONTEXT (so you target the real gap, not generic ML):
- System: a local pipeline detects the shuttle per-frame (WASB / TrackNet, MIT-licensed) ? a heuristic turns the trajectory into candidate rally windows ? today an optional Gemini "handoff" confirms rallies. We just measured a fully-local (no-AI) run vs Gemini on 3 matches: local emits far FEWER but MUCH LONGER windows (mean 65s vs 6.1s; one window spanned a whole 174s clip) ? it does NOT miss activity, it FAILS TO SEGMENT. Root cause (code-confirmed): a frame is "active" iff the shuttle is visible AND moving above a small per-second-normalized velocity threshold; active frames within a 2 s gap are merged; there is NO max-window cap and NO inactivity/boundary split. So "shuttle is moving" is conflated with "rally in progress," and dead-time + serves + knock-ups bridge rallies into mega-windows. Shuttle tracking itself is excellent (96?99% per-frame visibility).
- HARD CONSTRAINTS: weights/data/code must be MIT/Apache-2.0/BSD only (no viral/NC/custom licenses, reject Gemma-3/Llama-4-MAU-cap); we may NEVER train owned weights on paid-LLM (Gemini/OpenAI/Anthropic) outputs (terms/copyright), though paid LLMs may be used at inference; minors excluded from any training pool; runs on a single local GPU.
- WHAT WE ALREADY HAVE (do NOT re-survey the basics ? go deeper/newer): the architecture skeleton is known to be standard Temporal Action Localization/Segmentation (proposal+scorer) + late/score-level fusion + stacked generalization; in-domain prior art (MonoTrack, ShuttleSet, See et al. 2024/25 non-broadcast badminton rally start/end, Ghosh et al. point-segmentation, tennis/TT play-break state machines) is already cited. We have a working eval harness: leave-one-video-out (grouped by match), temporal-IoU F1 @0.5, F1@{0.1,0.25,0.5}, mAP@tIoU, segment-count-ratio, bootstrap 95% CIs, paired exact sign-test, Platt/isotonic calibration + Brier/ECE, a tiny numpy LogisticRegression scorer over per-window feature columns, an experiment leaderboard, and an "eval?serve" guard (a cue measured +0.074 F1 on permissive proposal windowing but ?0.008 on the coarser served windowing ? reverted). Available but default-OFF per-window signals: net-crossing count (rally gate), 5 person/court features, 3 audio-hit features.

THE FOUR DIMENSIONS:

1) WINDOWING & BOUNDARIES (Temporal Action Localization/Segmentation): beyond proposal+score ? what are the best *recent* methods for (a) turning a dense per-frame signal into well-segmented

actions, (b) explicitly controlling OVER-segmentation/merging, (c) boundary detection/regression and splitting merged segments? Cover e.g. MS-TCN/MS-TCN++, ASRF (action-boundary regression), BMN/BSN boundary-matching, ActionFormer/TriDet/TemporalMaxer (actionness + boundary heads), smoothing/boundary losses, and the standard metrics for over-segmentation (segmental F1@k, edit/Levenshtein score, mAP@tIoU). Which transfer to a 1-D shuttle-trajectory + auxiliary-cue setting on tiny data?

2) SHUTTLE-TRAJECTORY ? RALLY SIGNAL PROCESSING (racket-sports specific): how do published systems convert ball/shuttle tracking into rally/point segments and distinguish "in-play" from dead-time? Cover rally-state machines, serve/let detection, net-crossing & two-sided-occupancy cues, hit/contact (audio + kinematic) detection, and fps-invariant trajectory features. What features best separate "rally" from "shuttle merely moving"?

3) SMALL-DATA STRATEGIES: with ~6?16 labeled videos, what gives the most quality per label ? active learning (which videos/segments to label next; uncertainty/diversity/core-set), semi-/weakly-/point-supervised TAL (e.g. SF-Net, timestamp supervision), self-training/pseudo-labeling, and data-efficient TAL? Separately: the legally-clean ways to exploit a strong teacher (Gemini) at INFERENCE for weak/pseudo-labels or as an evaluation oracle WITHOUT its outputs becoming training data for owned weights ? what does the literature say about teacher-student / distillation boundaries and label-noise handling, and what governance distinctions matter?

4) EVALUATION METHODOLOGY & RIGOR at small n: trustworthy measurement with ~6?16 videos ? bootstrap/permutation tests, paired sign-tests, leave-one-group-out vs RLOOCV bias, calibration (reliability, ECE), the over-segmentation-blindness of plain tIoU, shadow/A-B evaluation, regression gates, and concrete pitfalls (label noise, annotator agreement/IAA, multiple-comparison/over-fitting traps, eval-vs-serve drift). What metrics + protocols should a small-data sports-segmentation leaderboard standardize on?

Deliverable: a structured, cited report with, per dimension, the 3?5 highest-leverage techniques that APPLY to our constrained setting (small data, local, license-clean, 1-D trajectory + cues + learned scorer), each with: what it is, why it fits our over-merge/small-data problem, the expected difficulty/payoff, and the citation. Call out which ideas are low-effort high-confidence wins vs research bets. Be explicit about anything unverified or license-incompatible.

Claim under review

"On THUMOS14, TemporalMaxer achieves 67.7% average mAP (mAP@0.5=71.8, @0.7=44.7), beating the ActionFormer baseline (66.8% avg) while using 16.4 GMACs and only 7.1M parameters versus ActionFormer's 45.3 GMACs / 29.3M params ? i.e. a parameter-free temporal block roughly 4x smaller than the attention model still wins on accuracy."

Source: https://arxiv.org/abs/2303.09055 (primary)
Supporting quote: "ActionFormer[60] I3D[7] 82.1 77.8 71.0 59.4 43.9 66.8 80 ...
Our(TemporalMaxer) I3D[7] 82.8 78.9 71.8 60.5 44.7 67.7 50"

Checklist

1. Is the claim actually supported by the quote, or is it an overreach/misread?
2. WebSearch for contradicting evidence ? does any credible source dispute or heavily qualify this?
3. Is the source quality sufficient for the claim's strength? (extraordinary claims need primary sources)
4. Is the claim outdated? (check dates ? old claims about fast-moving fields are suspect)
5. Is this a marketing claim / press release / cherry-picked benchmark / forum speculation?

refuted=true if: unsupported by quote / contradicted / low-quality source for strong claim / outdated / marketing fluff.
refuted=false ONLY if: claim is well-supported, current, and source quality matches claim strength.
Default to refuted=true if uncertain.

Structured output only. Evidence MUST be specific.

2. assistant (2026-06-16T03:24:36.087Z)

I'll verify this claim about TemporalMaxer vs ActionFormer on THUMOS14. Let me fetch the primary

source.

3. assistant (2026-06-16T03:24:38.657Z)

I'll fetch the primary source (the TemporalMaxer arXiv paper) to verify the exact numbers.

4. user (2026-06-16T03:24:43.374Z)

I appreciate your detailed request, but I cannot provide the specific THUMOS14 results table you're asking for. The web page content provided is only the arXiv abstract and metadata page for the TemporalMaxer paper?it does not include the actual results tables or comparative data.

To obtain the exact numbers you need (average mAP, mAP@0.5, mAP@0.7, GMACs, and parameter counts), you would need to:

1. Access the full PDF of the paper via the link provided: [View PDF](/pdf/2303.09055)
2. Check the THUMOS14 results section within the paper itself
3. Consult the GitHub repository mentioned: https://github.com/TuanTNG/TemporalMaxer

The abstract only states that TemporalMaxer "outperforms other state-of-the-art methods...while requiring significantly fewer parameters and computational resources," but the specific numerical comparisons are not included in this page excerpt.

5. assistant (2026-06-16T03:24:48.756Z)

The abstract page doesn't have the table. Let me fetch the full HTML version of the paper and search for corroborating sources.

6. user (2026-06-16T03:24:54.567Z)

I apologize, but I cannot extract the THUMOS14 results table you've requested from the provided content. The document appears to be a PDF file containing compressed/encoded stream data that isn't readable as plain text in this format.

To obtain the specific metrics you need (average mAP, mAP@various IoU thresholds, GMACs, and parameter counts for TemporalMaxer and ActionFormer), I recommend:

1. **Accessing the original paper directly** through arXiv or the conference proceedings
2. **Looking for the results table** in the main paper text (typically in the experimental results section)
3. **Checking supplementary materials** if detailed comparisons are provided there

If you can provide the paper in a readable text format or share the specific results table as text, I'd be happy to help extract and organize those exact numbers for you.

[Binary content (application/pdf, 1.1MB) also saved to C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\tool-results\webfetch-1781579965172-5ov5x2.pdf]

7. user (2026-06-16T03:25:01.757Z)

Web search results for query: "TemporalMaxer THUMOS14 average mAP 67.7 ActionFormer 66.8 parameters GMACs"

Links: [{"title":"TemporalMaxer Performance in the Face of Constraint: A Study in Temporal","url":"https://repository.tudelft.nl/file/File_7430e57c-b269-4010-a020-376bf7180e18"},{"title":"TemporalMaxer: Maximize Temporal Context with only Max Pooling","url":"https://arxiv.org/pdf/2303.09055"},{"title":". . Latest updates: h?ps://dl.acm.org/doi/10.1145/3652628.3652695 . .","url":"https://dl.acm.org/doi/pdf/10.1145/3652628.3652695"},{"title":"TemporalMaxer/README.md at master · TuanTNG/TemporalMaxer","url":"https://github.com/TuanTNG/TemporalMaxer/blob/master/README.md"},{"title":"GitHub - f1ibrahim-tmu/TemporalMaxer: TemporalMaxer: Maximize Temporal Context with only Max Pooling for Temporal Action Localization · GitHub","url":"https://github.com/f1ibrahim-tmu/TemporalMaxer"},{"title":"Benchmarking Data

Efficiency and Computational
Efficiency","url":"https://cveu.github.io/2023/papers/11.pdf"},{"title":"Adaptive Perception
Transformer for Temporal Action
Localization","url":"https://arxiv.org/pdf/2208.11908"},{"title":"Benchmarking Data Efficiency
and Computational Efficiency of Temporal Action Localization
Models","url":"https://arxiv.org/pdf/2308.13082"},{"title":"Temporal Zoom Networks: Distance
Regression and Continuous Depth for Efficient Action
Localization","url":"https://arxiv.org/pdf/2511.03943"},{"title":"Temporal Action Detection
Model Compression by Progressive Block Drop","url":"https://arxiv.org/pdf/2503.16916"}]

Based on the search results, here's what I found regarding your query:

TemporalMaxer achieved an average mAP of 67.7 on THUMOS14 with a kernel size of 3, and ActionFormer reached an average mAP of 66.8% on the THUMOS'14 dataset.

Regarding computational efficiency, TemporalMaxer outperforms ActionFormer with 2.8x fewer GMACs and 3x faster inference speed. Additionally, TemporalMaxer's backbone takes only 2.5 ms which is 8.0x faster than ActionFormer backbone at 20.1 ms.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

8. assistant (2026-06-16T03:25:04.042Z)

The PDF was saved locally. Let me read it directly to get the exact table, and check the GitHub README which often lists exact numbers.

9. user (2026-06-16T03:25:06.310Z)

pdftoppm failed: Command 'pdftoppm' not found or is in an unsafe location (current directory)

10. user (2026-06-16T03:25:12.148Z)

TemporalMaxer vs ActionFormer Comparison

Based on the README content provided, I can only extract limited comparison data:

Performance on THUMOS:
The document shows TemporalMaxer's results table:
- mAP@0.3: 82.8
- mAP@0.4: 78.9
- mAP@0.5: 71.8
- mAP@0.6: 60.5
- mAP@0.7: 44.7
- **Average mAP: 67.7**

Efficiency Claims:
The introduction mentions that TemporalMaxer achieves "2.8x fewer GMACs and 3x faster inference speed" compared to ActionFormer, and the backbone processes content in "2.5 ms which is incredibly 8.0x faster than ActionFormer backbone, 20.1 ms."

Limitation: The README does not include a direct side-by-side results table comparing ActionFormer and TemporalMaxer with specific parameter counts (in millions) for each model on THUMOS14. Only qualitative efficiency comparisons are provided.

11. assistant (2026-06-16T03:25:16.658Z)

The PDF can't be rendered (no pdftoppm). But I now have strong corroboration from two independent sources (GitHub README and the search summary) for the TemporalMaxer side. Let me verify the contested specifics: the ActionFormer per-threshold numbers (82.1/77.8/71.0/59.4/43.9/66.8) and especially the GMACs/parameter counts (16.4 GMACs / 7.1M params vs 45.3 GMACs / 29.3M params). The "2.8x fewer GMACs" claim from the README needs

checking against the specific "16.4 vs 45.3" numbers. Let me find a source with the exact computational table.

12. user (2026-06-16T03:25:19.731Z)

REDIRECT DETECTED: The URL redirects to a different host.

Original URL: https://ar5iv.org/abs/2303.09055
Redirect URL: https://ar5iv.labs.arxiv.org/abs/2303.09055
Status: 301 Moved Permanently

To complete your request, I need to fetch content from the redirected URL. Please use WebFetch again with these parameters:
- url: "https://ar5iv.labs.arxiv.org/abs/2303.09055"
- prompt: "Find the table that reports GMACs (or MACs/GFLOPs) and number of parameters for TemporalMaxer versus ActionFormer on THUMOS14. Report the exact GMACs and parameter count (in millions) for each model. Also report the full THUMOS14 mAP table: for ActionFormer and TemporalMaxer give mAP@0.3, 0.4, 0.5, 0.6, 0.7 and average mAP."

13. user (2026-06-16T03:25:23.480Z)

Web search results for query: "TemporalMaxer 7.1M parameters 16.4 GMACs ActionFormer 29.3M 45.3 GMACs THUMOS14"

Links: [{"title":"TemporalMaxer Performance in the Face of Constraint: A Study in Temporal","url":"https://repository.tudelft.nl/file/File_7430e57c-b269-4010-a020-376bf7180e18"},{"title":"TemporalMaxer/README.md at master · TuanTNG/TemporalMaxer","url":"https://github.com/TuanTNG/TemporalMaxer/blob/master/README.md"},{"title":"GitHub - f1ibrahim-tmu/TemporalMaxer: TemporalMaxer: Maximize Temporal Context with only Max Pooling for Temporal Action Localization · GitHub","url":"https://github.com/f1ibrahim-tmu/TemporalMaxer"},{"title":"GitHub - TuanTNG/TemporalMaxer: TemporalMaxer: Maximize Temporal Context with only Max Pooling for Temporal Action Localization","url":"https://github.com/TuanTNG/TemporalMaxer"},{"title":"TemporalMaxer: Maximize Temporal Context with only Max Pooling","url":"https://arxiv.org/pdf/2303.09055"},{"title":"Temporal Action Localization with Cross Layer Task Decoupling and Refinement","url":"https://arxiv.org/pdf/2412.09202"},{"title":"Benchmarking Data Efficiency and Computational Efficiency of Temporal Action Localization Models","url":"https://arxiv.org/pdf/2308.13082"},{"title":"Temporal Zoom Networks: Distance Regression and Continuous Depth for Efficient Action Localization","url":"https://arxiv.org/pdf/2511.03943"},{"title":"Benchmarking Data Efficiency and Computational Efficiency","url":"https://cveu.github.io/2023/papers/11.pdf"},{"title":"Temporal Action Detection Model Compression by Progressive Block Drop","url":"https://arxiv.org/pdf/2503.16916"}]

Based on the search results, I found relevant information comparing these models:

TemporalMaxer has 7.1M parameters with 30ms inference time, while ActionFormer has 29.3M parameters with 52ms inference time, according to a comparison table from a temporal action localization paper.

Regarding computational complexity, TemporalMaxer is effective at modeling temporal contexts and outperforms ActionFormer with 2.8x fewer GMACs. This aligns with the computational efficiency differences you mentioned in your query.

The search results also show that both models have been evaluated on the THUMOS14 dataset, which is a standard benchmark for temporal action localization tasks.

The specific numbers you provided (7.1M parameters with 16.4 GMACs for TemporalMaxer and 29.3M with 45.3 GMACs for ActionFormer) appear to be from technical papers comparing these models' computational efficiency and architectural complexity on the THUMOS14 dataset.

REMINDER: You MUST include the sources above in your response to the user using markdown

hyperlinks.

14. assistant (2026-06-16T03:25:25.653Z)

Good progress. The parameter counts (7.1M vs 29.3M) are confirmed. Let me fetch the ar5iv HTML version to get the exact GMACs numbers and the full ActionFormer mAP table.

15. user (2026-06-16T03:25:31.109Z)

THUMOS14 Results Comparison

Full mAP Table (Table 1)

For the two models of interest:

ActionFormer: mAP@0.3: 82.1, mAP@0.4: 77.8, mAP@0.5: 71.0, mAP@0.6: 59.4, mAP@0.7: 43.9, Average: 66.8

TemporalMaxer: mAP@0.3: 82.8, mAP@0.4: 78.9, mAP@0.5: 71.8, mAP@0.6: 60.5, mAP@0.7: 44.7, Average: 67.7

Computational Efficiency (Table 5)

The ablation study table provides efficiency metrics:

Transformer (ActionFormer baseline): 45.3 GMACs, 29.3M parameters

TemporalMaxer: 16.4 GMACs, 7.1M parameters

This represents approximately 2.8x fewer computational operations and 4.1x fewer parameters for TemporalMaxer compared to the Transformer-based ActionFormer approach.

16. assistant (2026-06-16T03:25:41.060Z)

I now have full verification from the primary source (ar5iv HTML rendering of arXiv:2303.09055), corroborated by the GitHub README and multiple search summaries.

Let me check each numeric assertion in the claim against the verified table values:

mAP table (Table 1):
- TemporalMaxer avg mAP = 67.7 ?
- TemporalMaxer mAP@0.5 = 71.8 ?
- TemporalMaxer mAP@0.7 = 44.7 ?
- ActionFormer avg mAP = 66.8 ?
- TemporalMaxer (67.7) beats ActionFormer (66.8) ?

Computational efficiency (Table 5):
- TemporalMaxer = 16.4 GMACs, 7.1M params ?
- ActionFormer/Transformer = 45.3 GMACs, 29.3M params ?

Derived claims:
- "roughly 4x smaller": 29.3M / 7.1M = 4.13x ? accurate ?
- "parameter-free temporal block" ? TemporalMaxer replaces the transformer with max pooling (parameter-free), accurate ?
- "still wins on accuracy" ? 67.7 > 66.8 ?

One minor nuance: the GMACs/param table (Table 5) is the paper's own ablation comparing its "Transformer" baseline (which IS ActionFormer's architecture, since TemporalMaxer is built on ActionFormer's codebase) vs the max-pooling block. The supporting quote provided in the claim only covers Table 1 (mAP), but the efficiency numbers (16.4/7.1 vs 45.3/29.3) are independently verified from Table 5 of the same primary paper. Every number in the claim matches the primary source exactly.

The "I3D" feature backbone qualifier is correct context (both rows use I3D features). The claim is fully supported, current for the 2022-2025 window, and sourced to the primary arXiv paper.

17. user (2026-06-16T03:25:50.683Z)

Structured output provided successfully

18. assistant (2026-06-16T03:25:55.956Z)

Verification complete. The claim is fully supported by the primary source and not refuted.

Result: refuted=false (high confidence)

All numbers in the claim match arXiv:2303.09055 exactly:
- Table 1 (mAP): TemporalMaxer 67.7 avg / 71.8@0.5 / 44.7@0.7 vs ActionFormer 66.8 avg ? verified
- Table 5 (efficiency): TemporalMaxer 16.4 GMACs / 7.1M params vs ActionFormer 45.3 GMACs / 29.3M params ? verified
- Derived "~4x smaller" (4.13x) and "parameter-free block still wins on accuracy" ? both accurate

Corroborated across three independent channels (ar5iv HTML of the primary paper, the official GitHub README's "2.8x fewer GMACs" statement, and separate search summaries confirming 7.1M vs 29.3M). The only nuance is that the mAP and efficiency figures come from two different tables in the same paper, with ActionFormer's architecture labeled "Transformer" in the efficiency ablation ? consistent, not a misread.